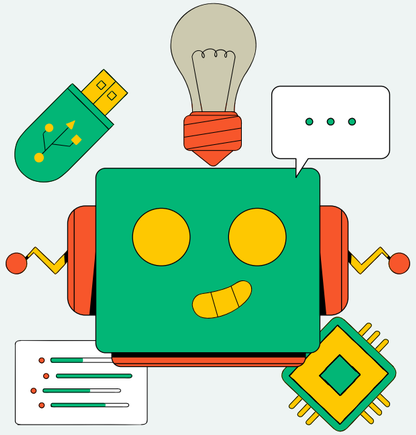
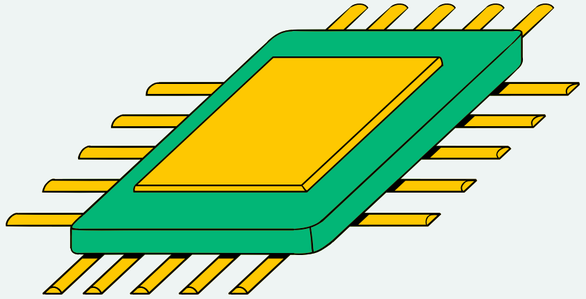


FAST ANOMALOUS TRAFFIC DETECTION SYSTEM FOR SECURE VEHICULAR COMMUNICATIONS

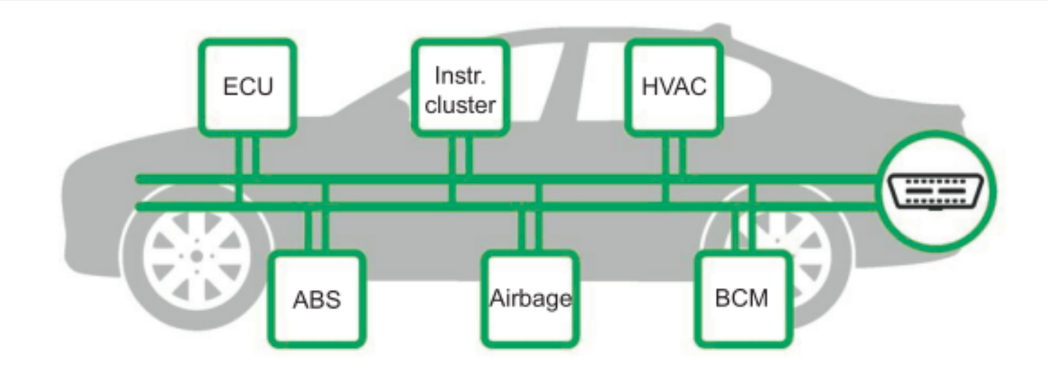
PRESENTED BY:

GROUP 9



PROBLEM STATEMENT AND RESEARCH OBJECTIVE

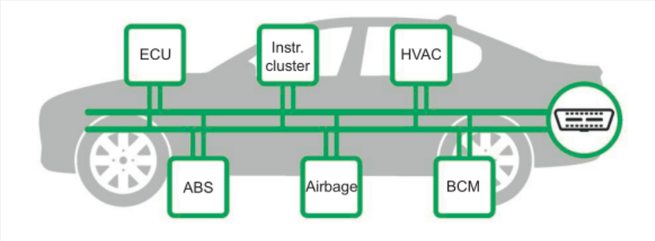
Modern vehicles act as Cyber-Physical Systems (CPS) utilizing the Controller Area Network (CAN) for internal communication



The Problem:

1. CAN lacks built-in security features:
2. No Authentication (Any node can broadcast)
3. No Encryption (Traffic is plaintext).
4. Vulnerability to Message Injection.

WHY IN-VEHICLE CAN SECURITY MATTERS



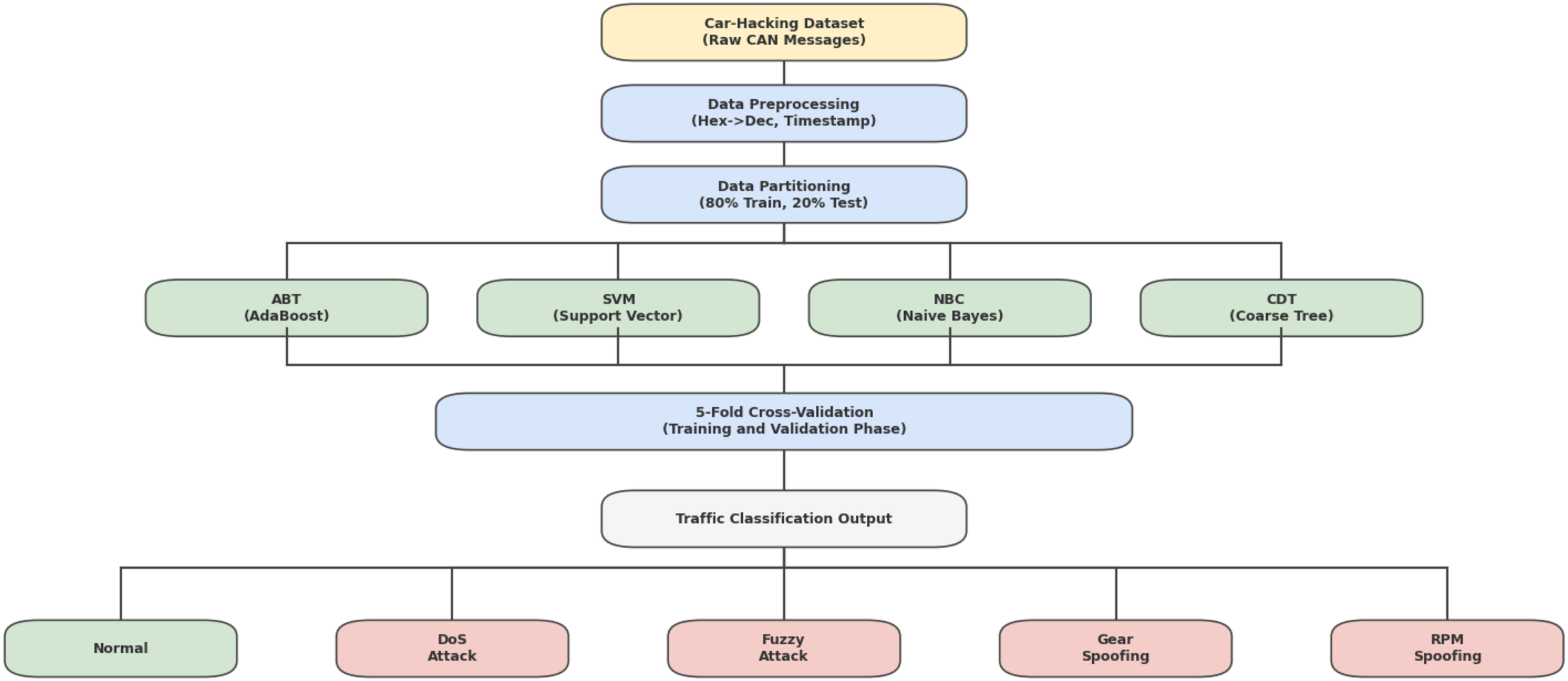
Modern vehicles contain many ECUs communicating over Controller Area Network (CAN)

CAN was not designed with security (limited authentication/integrity protection)

Attackers can inject messages (physical via OBD / remote via wireless interfaces)

Impact: incorrect ECU decisions : safety risk, loss of integrity, possible system failure

SYSTEM ARCHITECTURE



DATASET AND ATTACK TYPES (CAR-HACKING-2018)

Source: Car-Hacking-2018 Dataset (HCRL).

Features: Timestamp, CAN ID, DLC (Data Length Code), and Data Bytes.

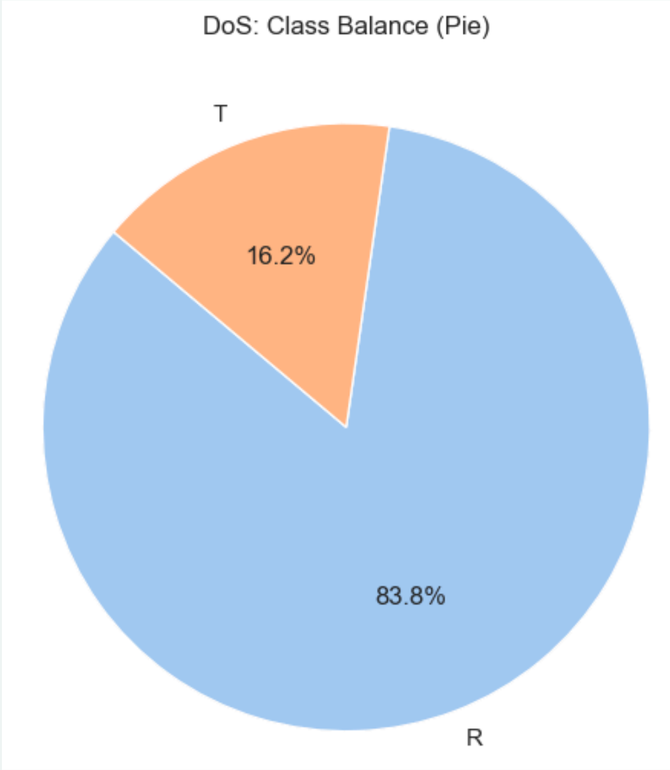
The raw dataset suffers from significant imbalance (Normal traffic dominates).

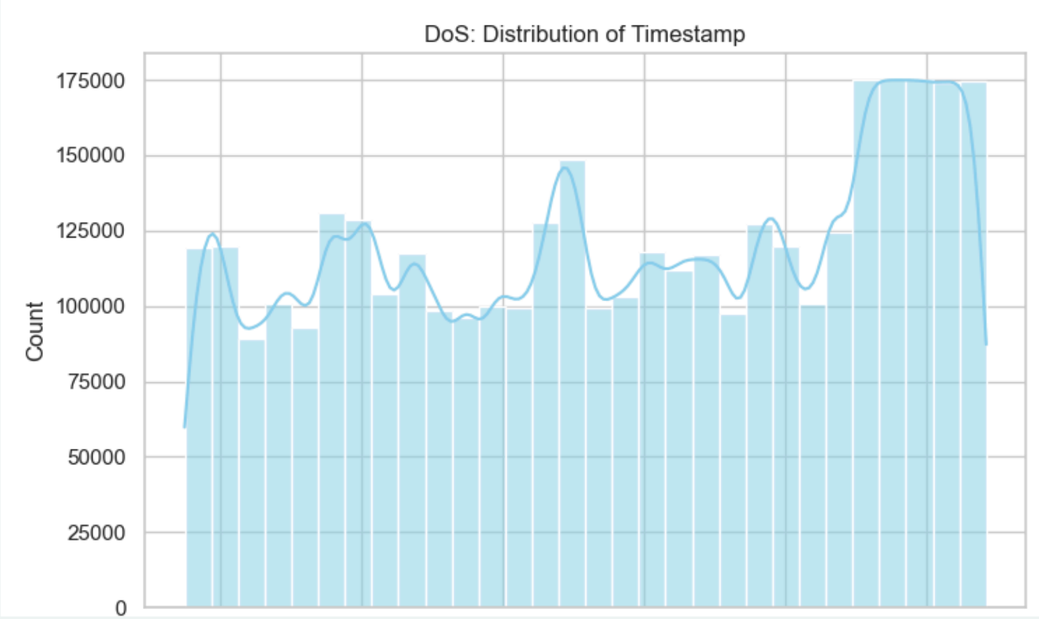
The dataset covers DoS, Fuzzy, and Gear/RPM spoofing attacks.

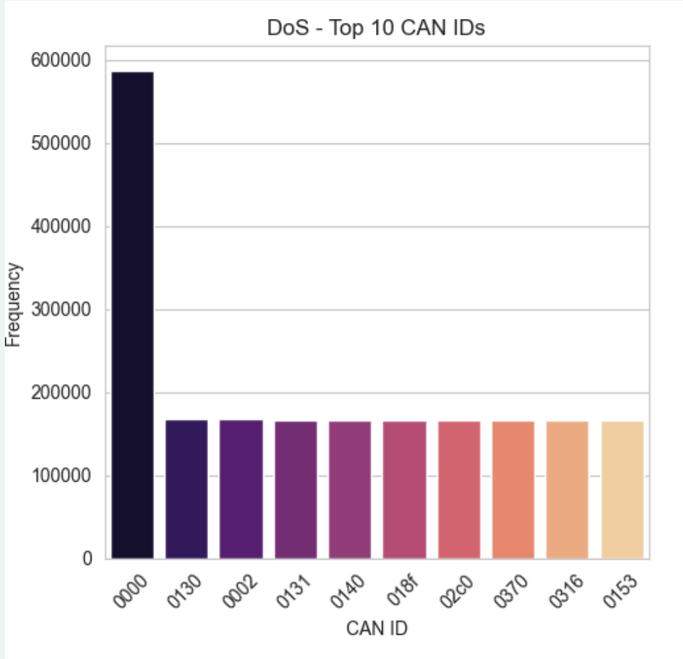
Targeted Attack:

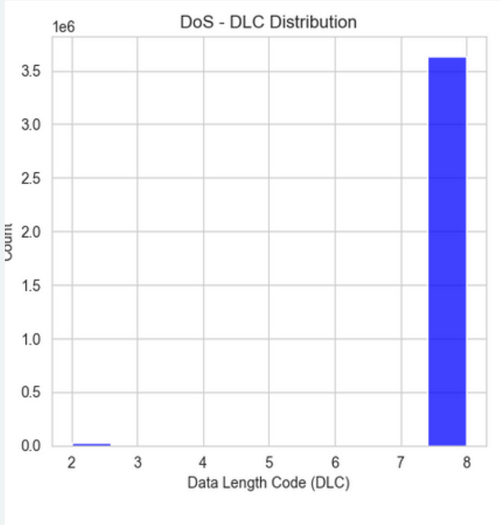
1. DoS: Flooding the bus to disable the operations.
2. Fuzzy: Injecting random signals to confuse ECUs.
3. Spoofing (RPM & Gear): Falsifying dashboard/engine data.

DATASET EXPLORATION – DOS ATTACK ANALYSIS

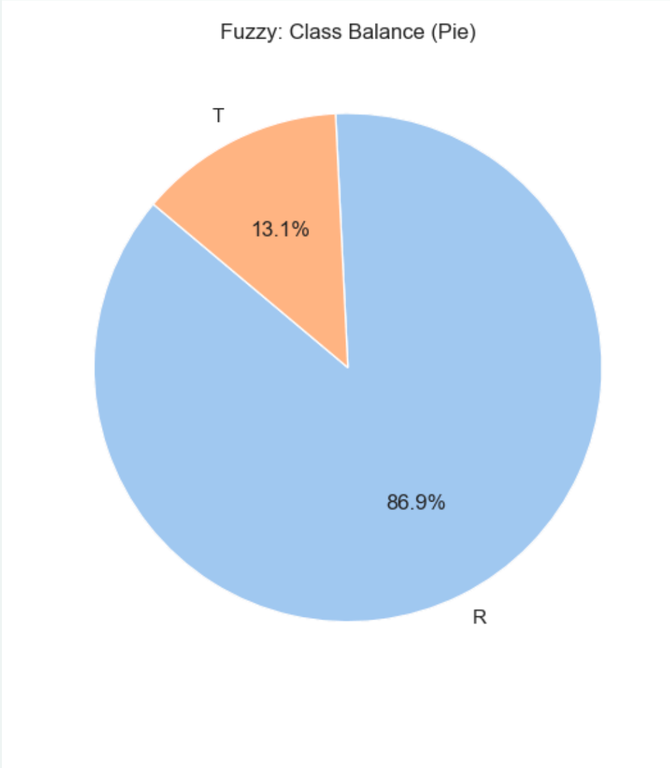


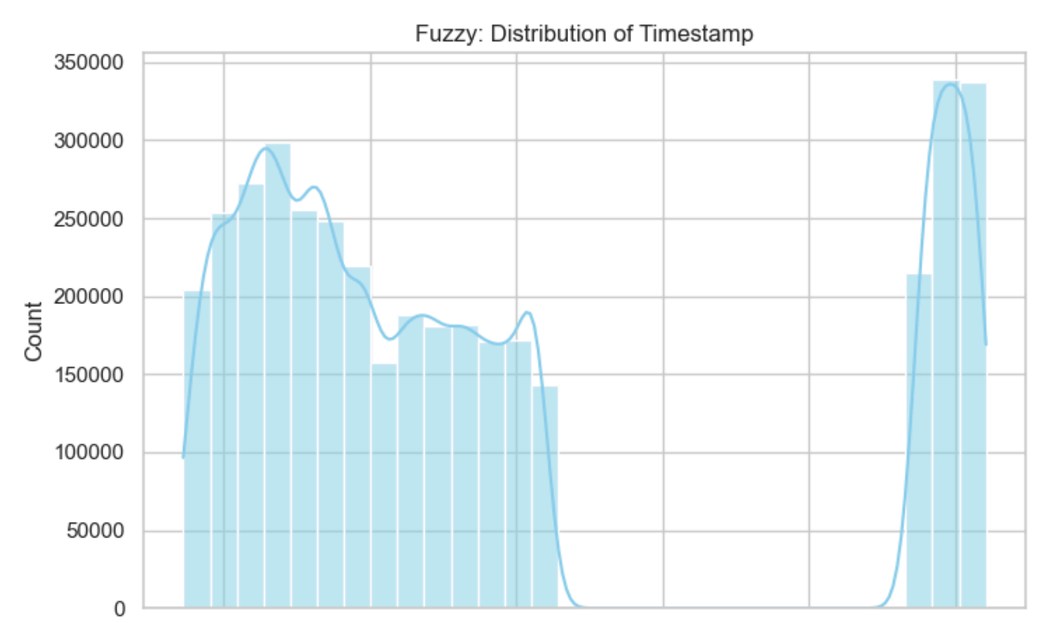


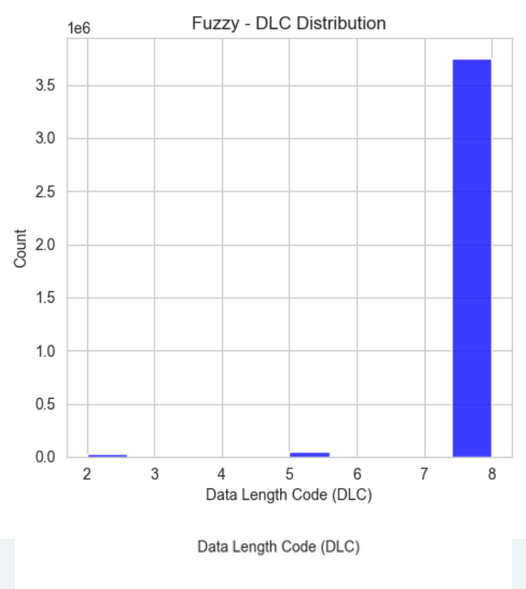


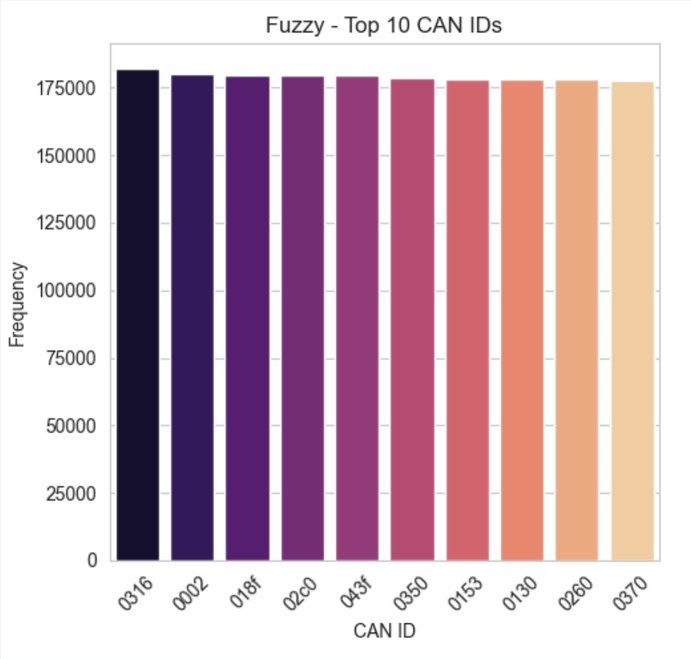


DATASET EXPLORATION – FUZZY ATTACK ANALYSIS

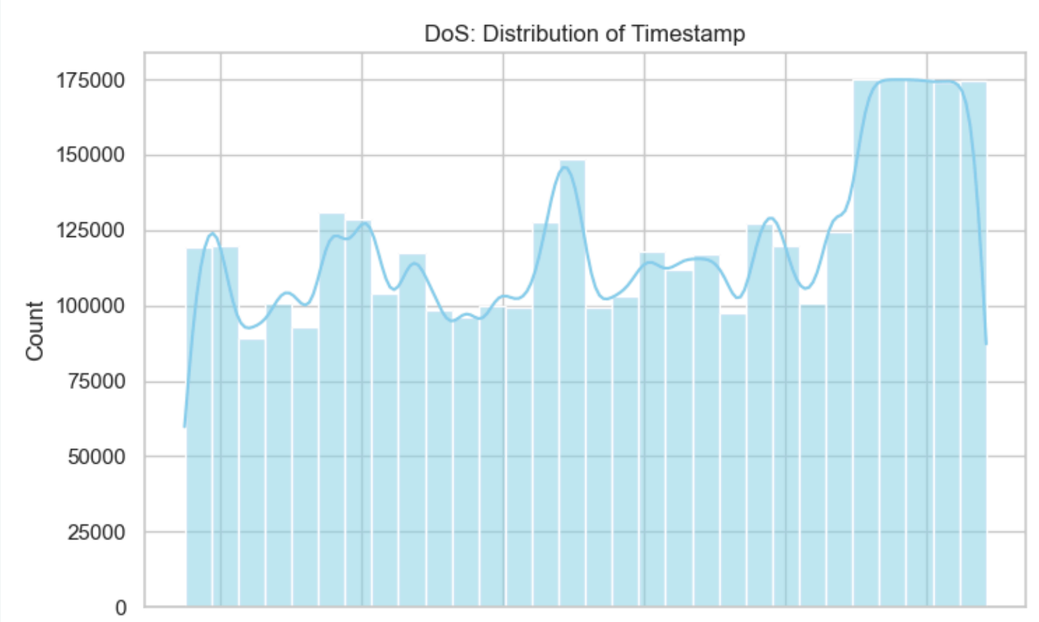


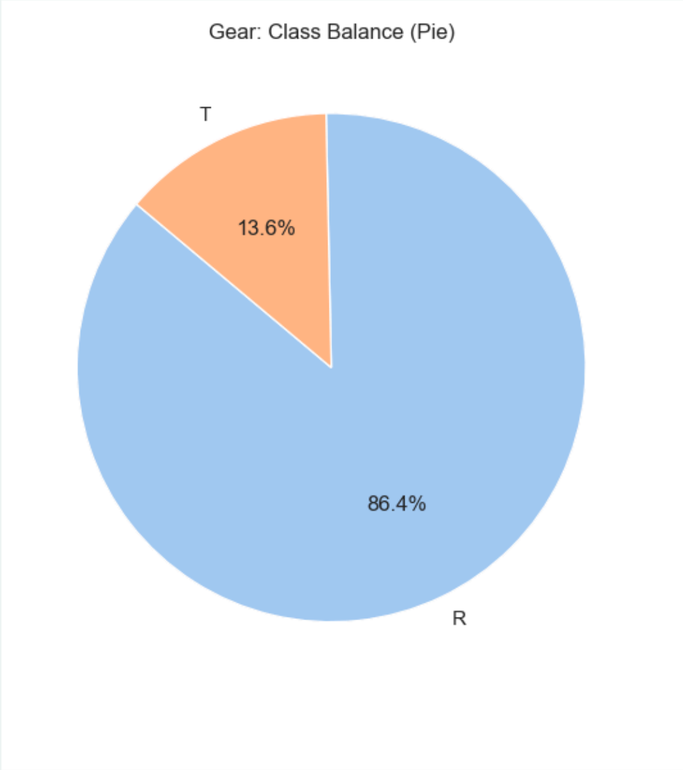


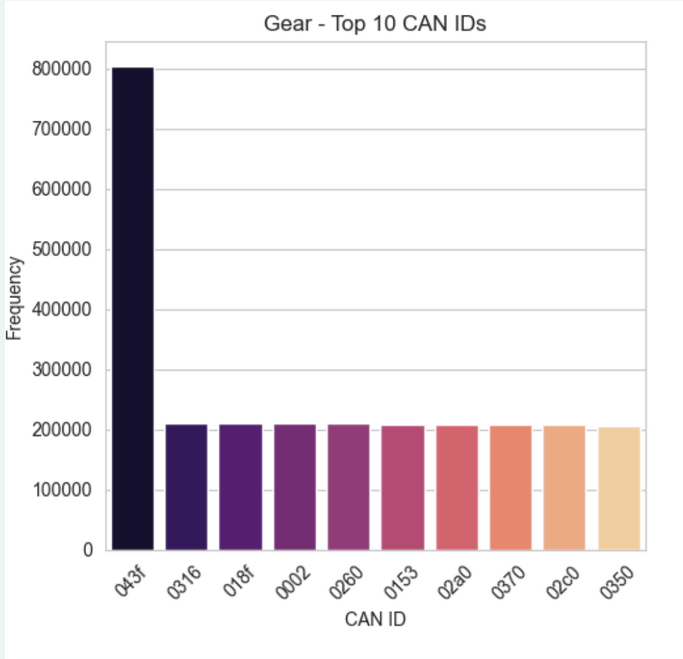


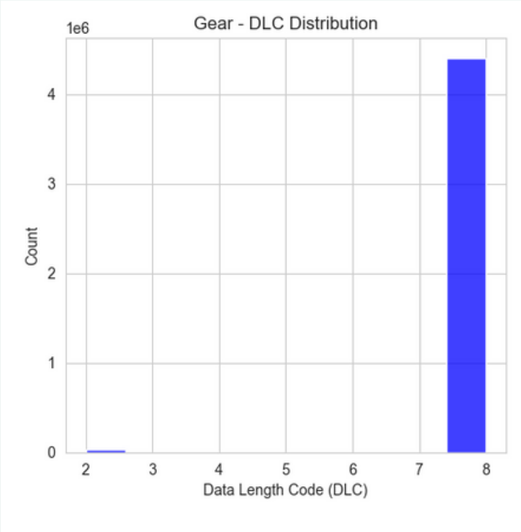


DATASET EXPLORATION – GEAR SPOOFING ANALYSIS

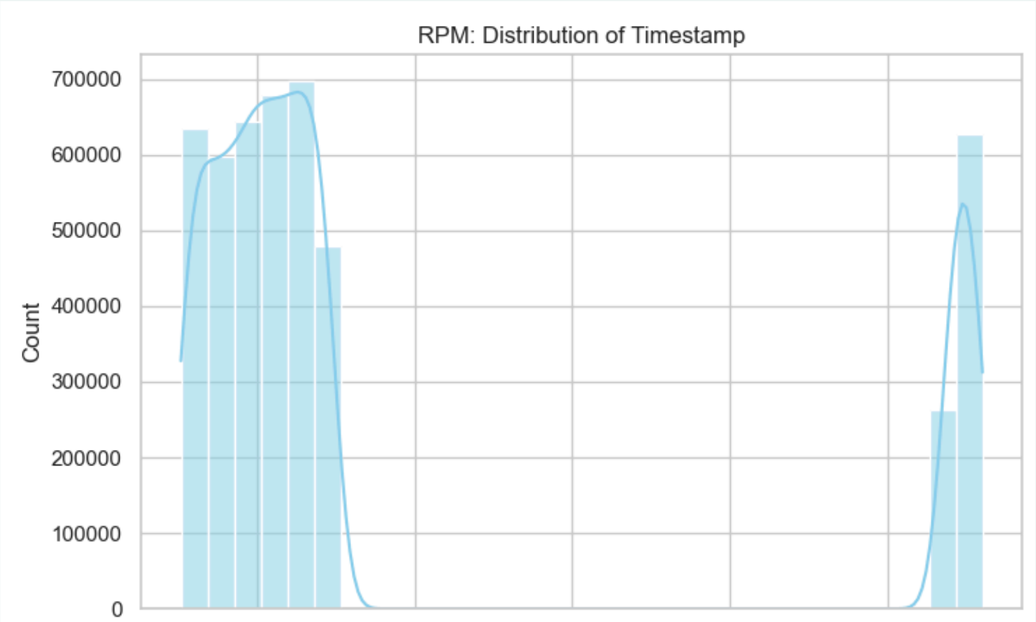


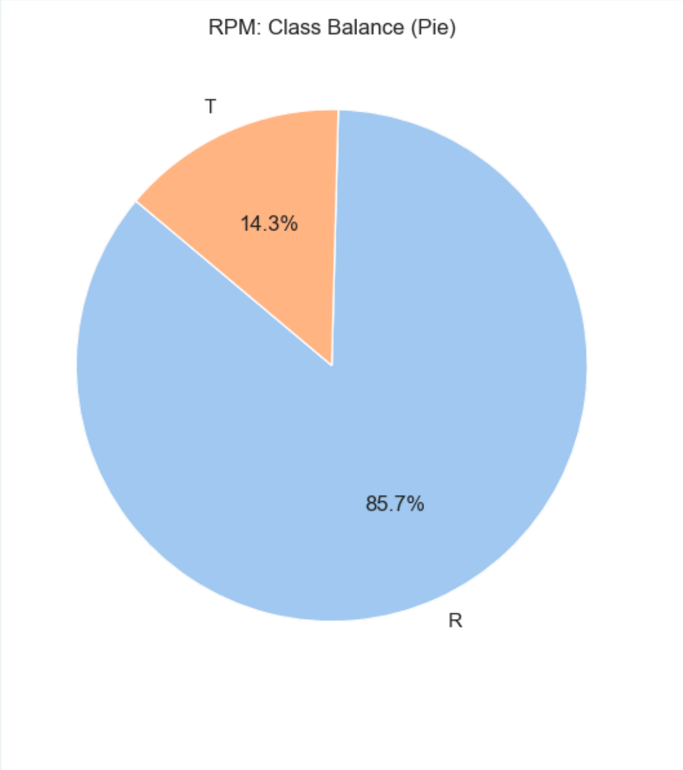


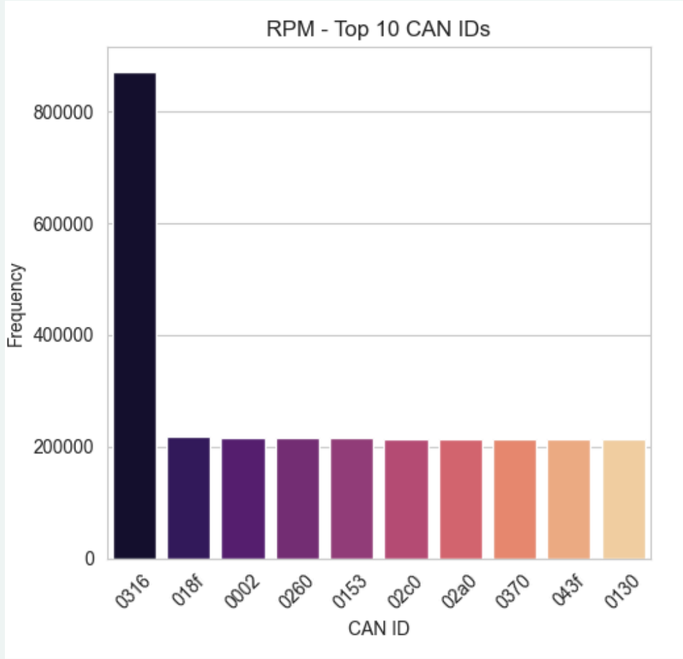


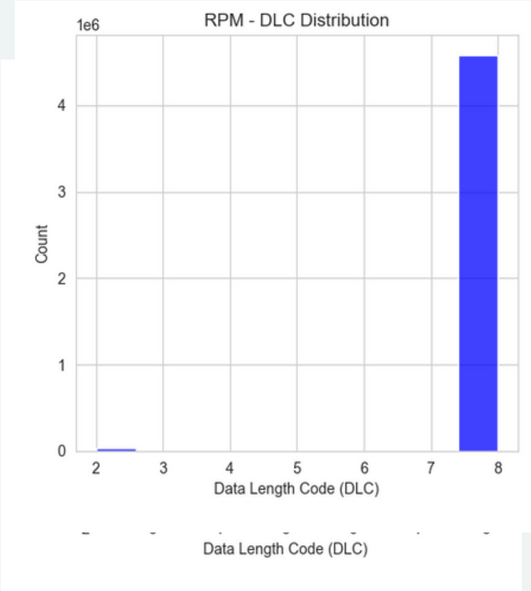


DATASET EXPLORATION – RPM SPOOFING ANALYSIS

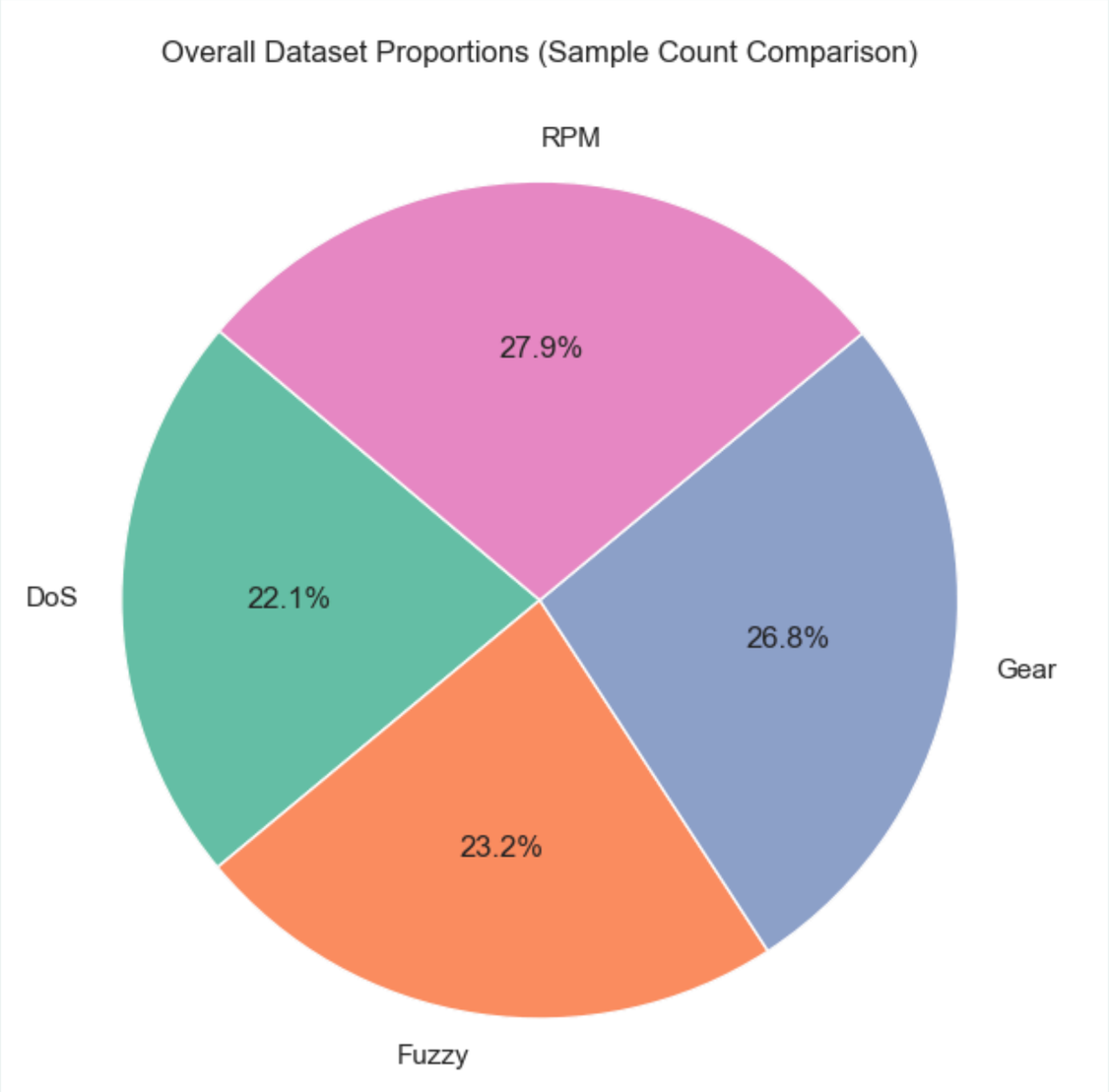








OVERALL DATASET DISTRIBUTION



DATASET EXPLORATION

01

LABEL DISTRIBUTION

Each attack file contains a mix of normal vs injected frames, providing a clear view of the balance between benign data and attack vectors.

02

PAYLOAD AND DLC

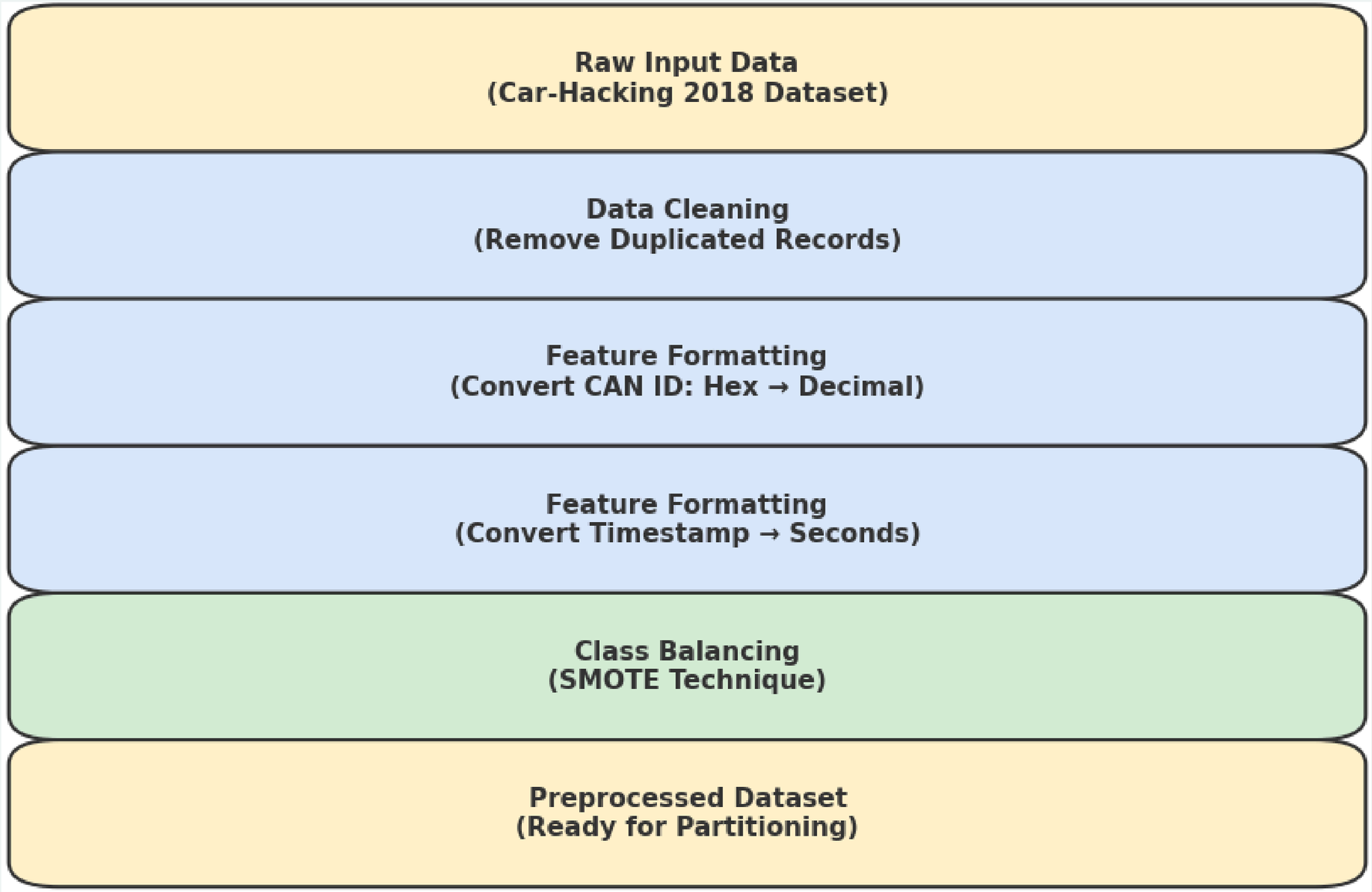
The DLC distribution analysis indicates that most frames utilize a dominant payload length (commonly DLC=8) throughout the captured traffic.

03

TRAFFIC PATTERNS

CAN ID distribution shows a small set of frequent IDs dominating traffic, while timestamp plots reveal non-uniform density and potential outliers.

PREPROCESSING AND FEATURE ENGINEERING



METHODOLOGY AND IMPLEMENTATION

Environment: Python (Scikit-learn, Pandas, NumPy).

Preprocessing Steps:
Handling missing values and duplicates.
Simulating real-world noise for robustness.

Train/Test Split:
80% Training: 2.3M samples
20% Testing: 597k samples

The dataset covers DoS, Fuzzy, and Gear/RPM spoofing attacks.

MODELS DEPLOYED:

- Naive Bayes
- Logistic Regression
- Bagging (DT)
- Boosting (AdaBoost)
- Decision Tree
- SVM (RBF kernel)
- Random Forest
- Stacking (RF+NB)

MODELS DEPLOYED:

Naive Bayes (NB): A fast probabilistic baseline that assumes features act independently; useful as a lightweight reference for CAN anomaly detection.

Logistic Regression (LR): A simple linear classifier that separates classes using weighted features; effective when preprocessing makes patterns more separable.

Decision Tree (DT): A rule-based model that splits on feature thresholds; interpretable and captures non-linear attack patterns, but can overfit if unconstrained.

Random Forest (RF): An ensemble of many decision trees (bagging) that reduces overfitting and improves robustness across different attack behaviors.

MODELS DEPLOYED:

SVM (Kernel = RBF): A non-linear margin-based classifier that maps features to a higher-dimensional space to separate complex attack patterns.

Bagging (DT): Multiple decision trees trained on bootstrapped samples; reduces variance and stabilizes predictions for noisy CAN traffic.

Boosting (AdaBoost): Sequentially combines weak learners by focusing more on previously misclassified frames to improve detection accuracy.

Stacking (RF + NB): A hybrid ensemble that merges Random Forest and Naïve Bayes outputs (meta-learning) to leverage both robustness and speed.

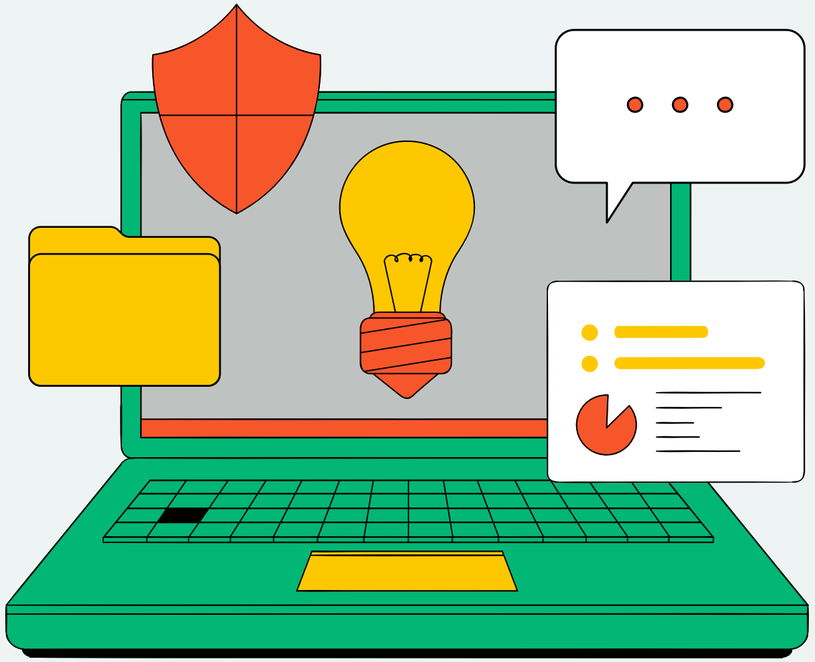
EXPERIMENTAL SETUP

Added noise to data for realism

Training Loop: Evaluated each model on test set

Visualizations Generated: Confusion Matrices

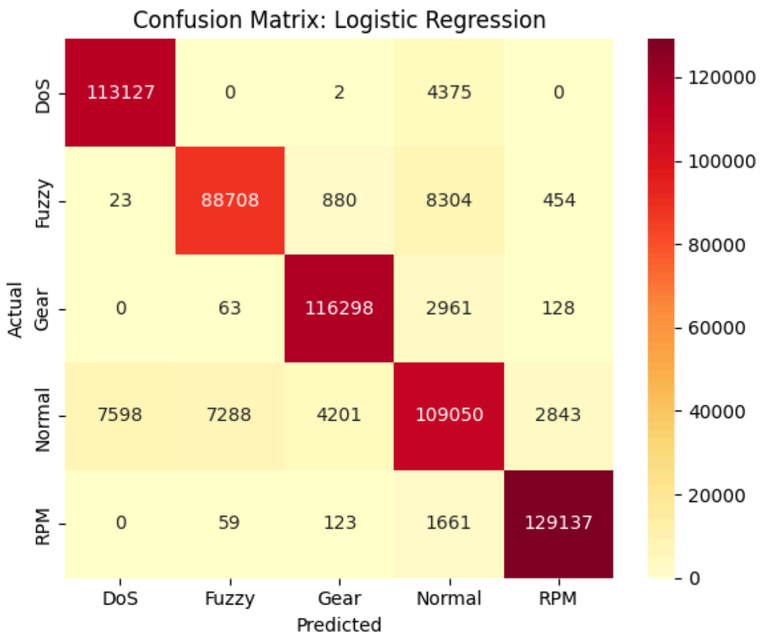
Metrics Calculation: Accuracy, Precision, Recall, F1-Score, Total Inference Time , Inference Time (us/sample)

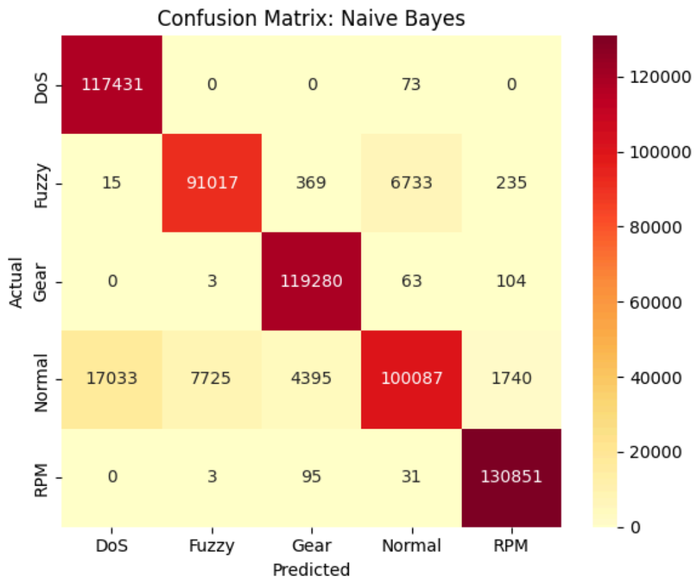


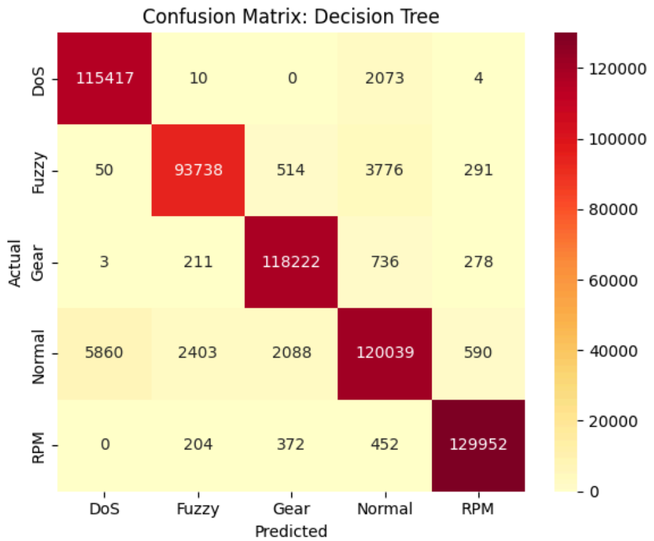
PERFORMANCE EVALUATION – OVERALL RESULTS

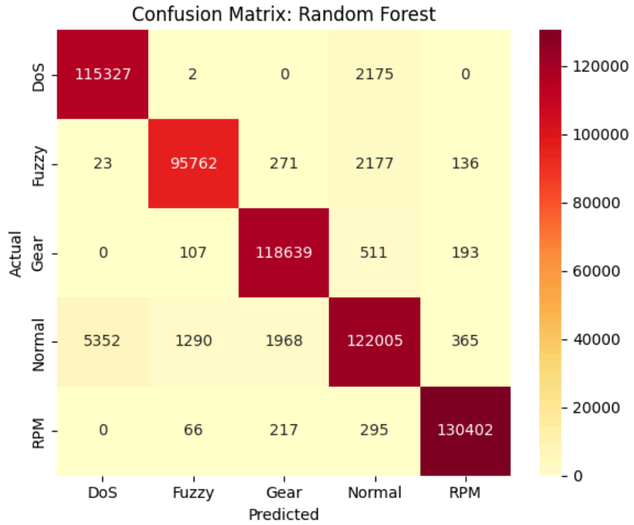
Model	Accuracy	Precision	Recall	F1-Score	Inference Time (Total)	Time per Sample (μs)
Random Forest	0.9746	0.9746	0.9746	0.9745	3.42s	5.73
SVM (Kernel=RBF)	0.9739	0.974	0.9739	0.9738	166.49s	278.75
Stacking (RF+NB)	0.9706	0.9705	0.9706	0.9705	1.73s	2.9
Bagging (DT)	0.9706	0.9706	0.9706	0.9705	1.20s	2
Decision Tree	0.9667	0.9666	0.9667	0.9665	0.11s	0.19
Naive Bayes	0.9353	0.9368	0.9353	0.9332	0.36s	0.61
Logistic Regression	0.9314	0.9306	0.9314	0.9309	0.04s	0.07
Boosting (AdaBoost)	0.8891	0.9065	0.8891	0.8903	6.11s	10.22

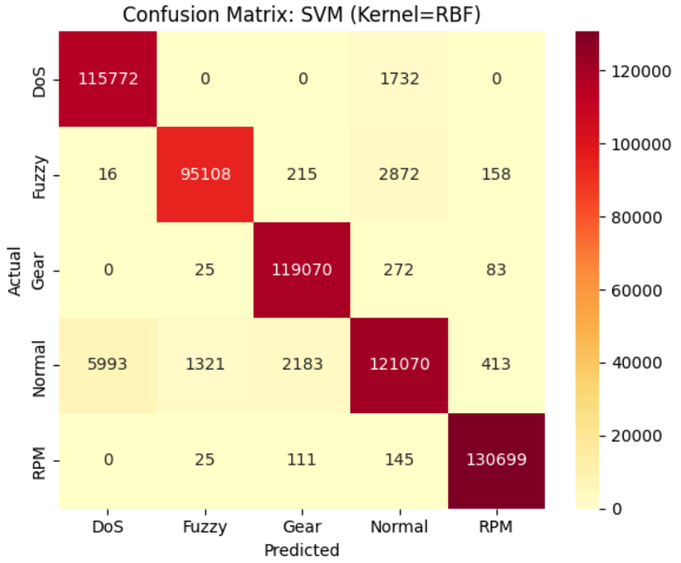
CONFUSION MATRIX ANALYSIS CLASSICAL ML MODELS



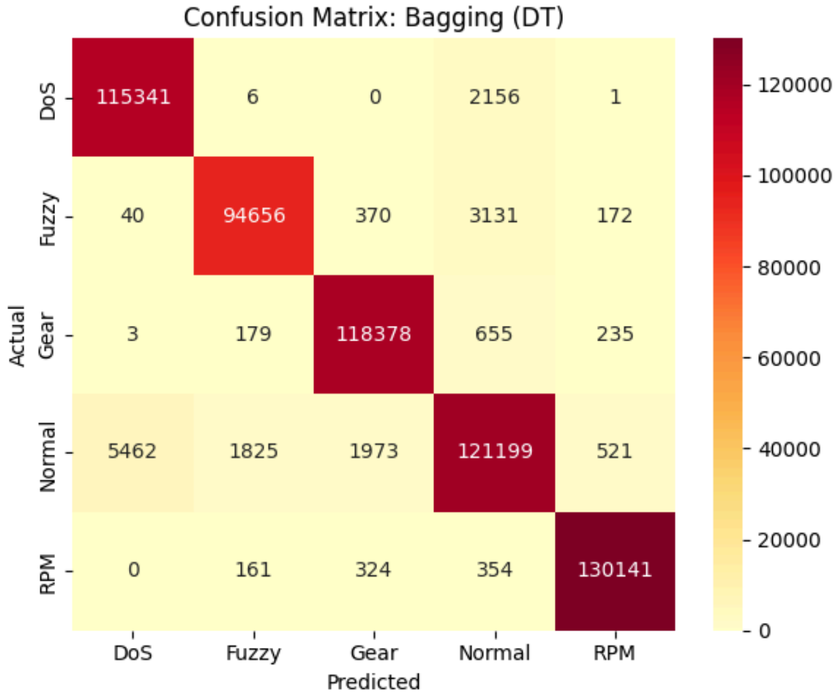


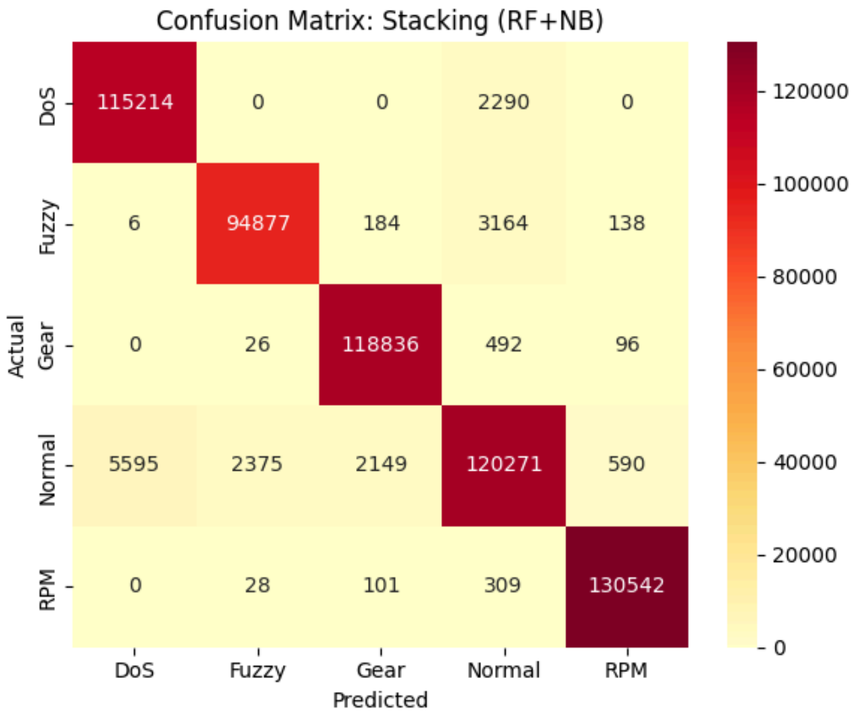


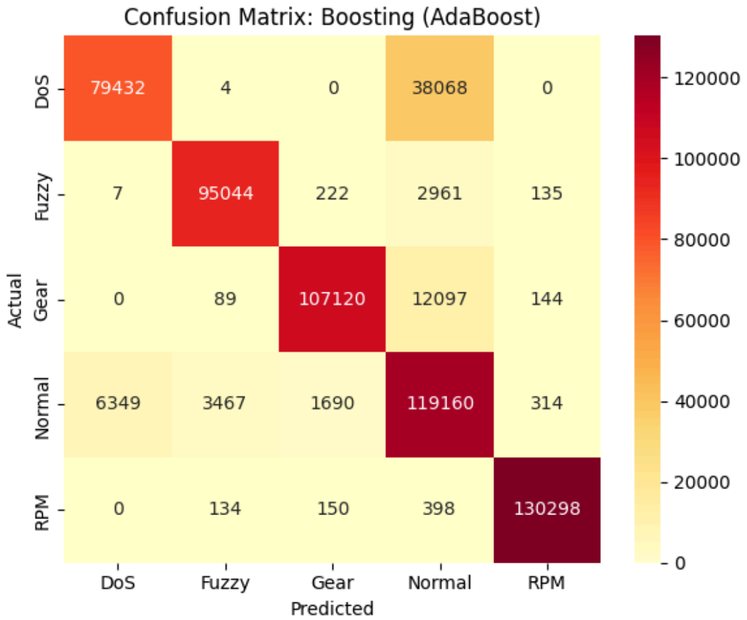




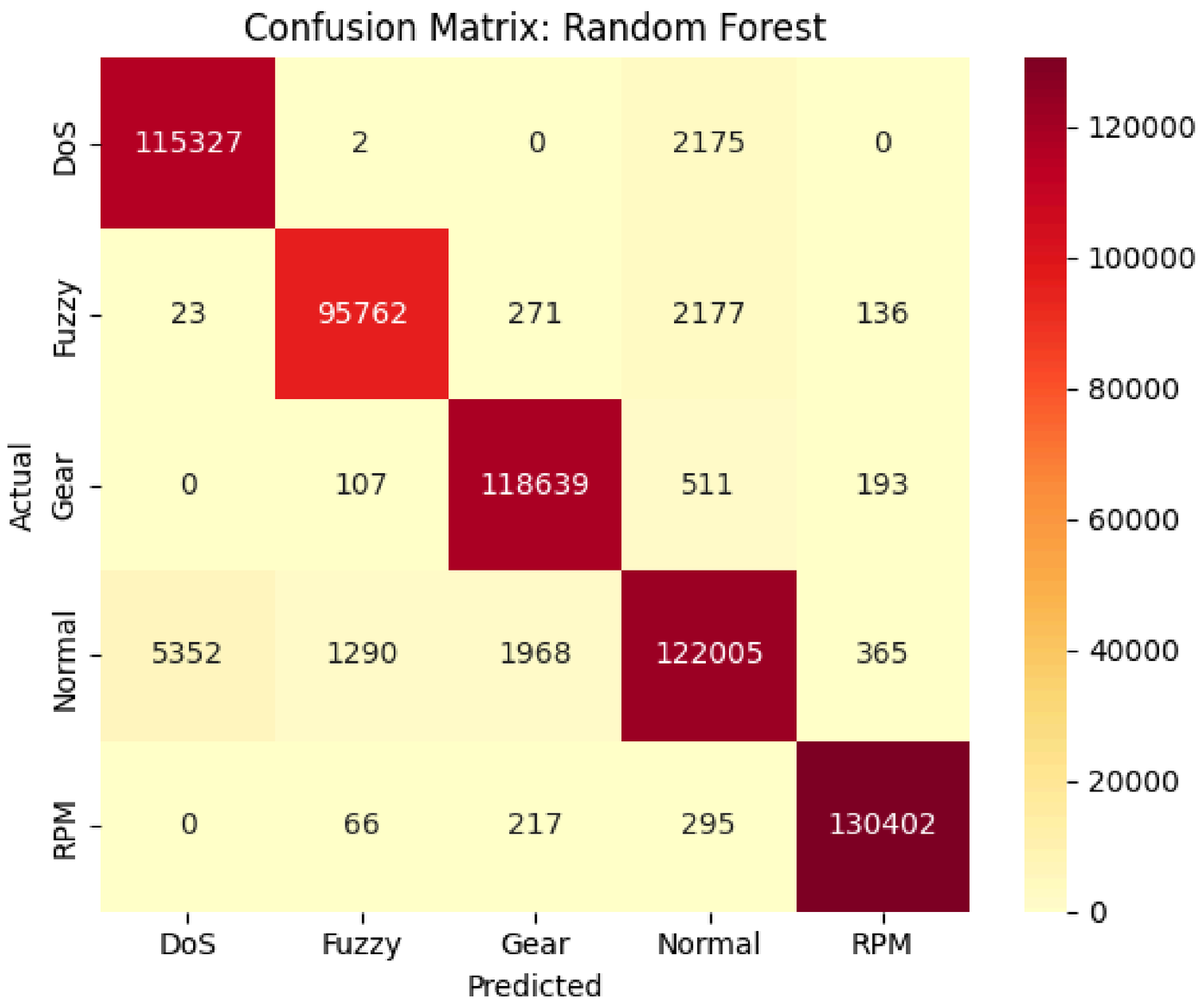
CONFUSION MATRIX ANALYSIS ENSEMBLE TECHNIQUES







CONFUSION MATRIX ANALYSIS BEST ACCURACY

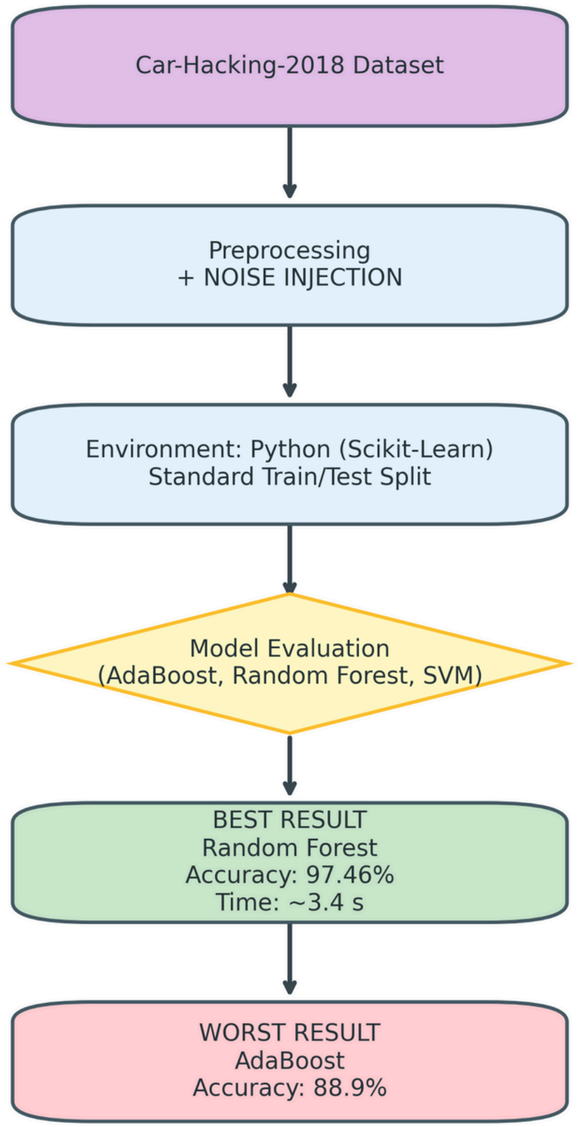


COMPARATIVE ANALYSIS (OURS VS. ORIGINAL PAPER)

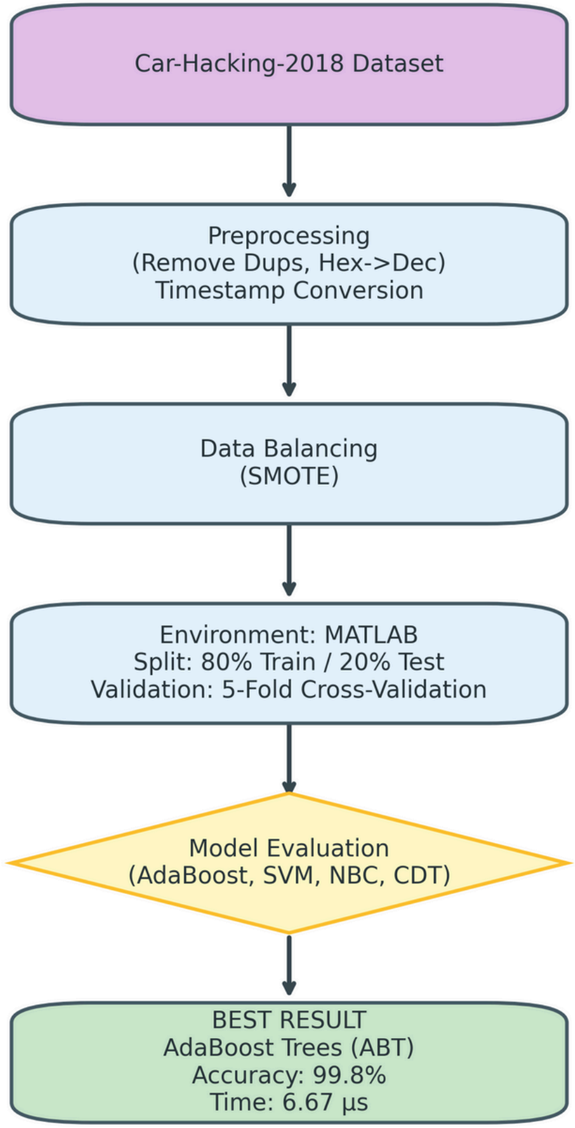
Feature	Original Paper Result (Best Model)	Our Experimental Result (Best Model)
Best Model	AdaBoost Trees (ABT)	Random Forest
Accuracy	99.80%	97.46%
Inference Time	6.67 μs (per sample)	~5.73 μs (per sample)
Worst Model	Coarse Decision Tree (79.8%)	Boosting / AdaBoost (88.9%)
Environment	MATLAB (No Noise)	Python (Added Noise)

ORIGINAL RESEARCH VS. OUR IMPLEMENTATION

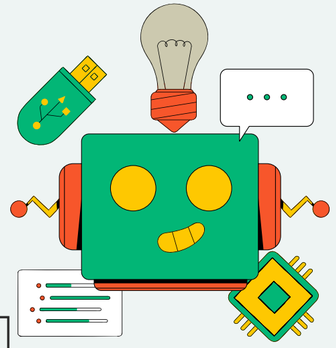
Our Extended Workflow



Original Research Paper Workflow



CONCLUSION



Random Forest achieved the best balance of accuracy (97.46%) and inference efficiency (~5.7 µs/sample).
SVM matched RF in accuracy but was computationally expensive (166s total).
AdaBoost underperformed in our setup, diverging from the original paper's claim of superior performance.
Confusion matrices revealed that fuzzy attacks remain the hardest to classify consistently.
Extended the original study by testing additional models (Logistic Regression, Bagging, Stacking).
Introduced noise injection to simulate real-world vehicular sensor variability.
Provided a direct comparative analysis between MATLAB-based and Python-based implementations.